

Module 4 Programming I



Java Swing Database Application

Objective:

To create a Java Swing application for managing student records in a MariaDB database. The application should allow users to insert, update, and delete student records.

Prerequisites:

- Basic understanding of Java programming.
- MariaDB installed on the local machine.
- MariaDB JDBC driver included in the project.

Instructions:

Part 1: Database Setup

1. Install MariaDB:

- Install MariaDB on your machine.
- Create a database named `University`.

2. Create Tables:

- Execute the following SQL queries to create the necessary tables:

```
CREATE TABLE Students ( studentID INT PRIMARY KEY, firstName VARCHAR(50), lastName VARCHAR(50) ); CREATE TABLE Courses ( courseID INT PRIMARY KEY, studentID INT, courseTitle VARCHAR(100), CONSTRAINT fk_student FOREIGN KEY (studentID) REFERENCES Students(studentID) );
```

Part 2: Java Swing Application

3. Download JDBC Driver:

- Download the MariaDB JDBC driver from <https://mariadb.com/downloads/connectors/connectors-data-access/java8-connector/>  (<https://mariadb.com/downloads/connectors/connectors-data-access/java8-connector/>).
- Add the JAR file to your project.

4. Create Java Swing Application:

- Use the provided Java code for the Swing application.
- Modify the database connection details in the code.

5. Insert Record Button:

- Implement the logic for the "Insert Record" button.
- When clicked, the application should prompt the user to enter a new student's ID, first name, and last name. It should then insert this record into the `Students` table.

6. Update Record Button:

- Implement the logic for the "Update Record" button.
- When clicked, the application should prompt the user to enter the student ID to update and the new first name and last name values. It should then update the corresponding record in the `Students` table.

7. Delete Record Button:

- Implement the logic for the "Delete Record" button.
- When clicked, the application should prompt the user to enter the student ID to delete. It should then delete the corresponding record from the `Students` table.

Grading Rubric:

• Insert Record (15 points):

- Correctly inserts a new record into the `Students` table.

- Handles input validation.
- **Update Record (15 points):**
 - Correctly updates an existing record in the `Students` table.
 - Handles input validation.
- **Delete Record (15 points):**
 - Correctly deletes a record from the `Students` table.
 - Handles input validation.
- **UI and Usability (5 points):**
 - The application should have a clear and user-friendly interface.
 - Properly formatted text fields and buttons.
- **Error Handling (5 points):**
 - Handles potential errors gracefully.
 - Provides informative error messages.

Submission:

- Submit the modified Java code.
- Include at least two screenshots demonstrating the application's functionality (inserting, updating, and deleting records).

Skeleton Code

```
import javax.swing.*;
import java.awt.*;
```

```
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class StudentDatabaseApp extends JFrame {

    // Components
    private JButton insertButton, updateButton, deleteButton;
    private JTextArea displayArea;

    // Database connection parameters
    private static final String DB_URL = "jdbc:mariadb://your_database_url:3306/your_database_name";
    private static final String DB_USER = "your_username";
    private static final String DB_PASSWORD = "your_password";

    public StudentDatabaseApp() {
        // Initialize components
        insertButton = new JButton("Insert Record");
        updateButton = new JButton("Update Record");
        deleteButton = new JButton("Delete Record");
        displayArea = new JTextArea(20, 40);

        // Set layout manager
        setLayout(new FlowLayout());

        // Add components to the frame
        add(insertButton);
        add(updateButton);
        add(deleteButton);
        add(new JScrollPane(displayArea));
    }
}
```

// Set up event listeners

```
insertButton.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        // Handle insert button click  
        insertRecord();  
    }  
});  
  
updateButton.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        // Handle update button click  
        // Implement logic to update a record in the database  
    }  
});  
  
deleteButton.addActionListener(new ActionListener() {  
    @Override  
    public void actionPerformed(ActionEvent e) {  
        // Handle delete button click  
        // Implement logic to delete a record from the database  
    }  
});
```

// Set frame properties

```
setTitle("Student Database App");  
setSize(500, 400);  
setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
setLocationRelativeTo(null);  
setVisible(true);  
}
```

// Method to handle inserting a record into the database

```
private void insertRecord() {
```

```
    try {
```

```
        // Establish a database connection
```

```
        Connection connection = DriverManager.getConnection(DB_URL, DB_USER, DB_PASSWORD);
```

```
        // Get user input for a new record (you need to customize this part)
```

```
        String studentID = JOptionPane.showInputDialog("Enter Student ID:");
```

```
        String firstName = JOptionPane.showInputDialog("Enter First Name:");
```

```
        String lastName = JOptionPane.showInputDialog("Enter Last Name:");
```

```
        // SQL query for inserting a new record into the Students table
```

```
        String sql = "INSERT INTO Students (studentID, FirstName, LastName) VALUES (?, ?, ?)";
```

```
        // Create a PreparedStatement
```

```
        PreparedStatement preparedStatement = connection.prepareStatement(sql);
```

```
        preparedStatement.setString(1, studentID);
```

```
        preparedStatement.setString(2, firstName);
```

```
        preparedStatement.setString(3, lastName);
```

```
        // Execute the query
```

```
        preparedStatement.executeUpdate();
```

```
        // Close resources
```

```
        preparedStatement.close();
```

```
        connection.close();
```

```
        // Display a success message
```

```
        JOptionPane.showMessageDialog(this, "Record inserted successfully!");
```

```
    } catch (SQLException ex) {
```

```
        // Handle database errors
```

```
        ex.printStackTrace();
```

```
        JOptionPane.showMessageDialog(this, "Error inserting record: " + ex.getMessage(), "Error", JOptionPane.ERROR_MESSAGE);
```

```
    }  
}  
  
public static void main(String[] args) {  
    // Register the JDBC driver (this should be done only once)  
    try {  
        Class.forName("org.mariadb.jdbc.Driver");  
    } catch (ClassNotFoundException e) {  
        e.printStackTrace();  
    }  
  
    SwingUtilities.invokeLater(new Runnable() {  
        @Override  
        public void run() {  
            new StudentDatabaseApp();  
        }  
    });  
}
```

Points 100

Submitting a file upload

Due	For	Available from	Until
Oct 4	Everyone	-	Oct 22 at 11:59pm